

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“Национальный исследовательский университет ИТМО”

Факультет прикладной информатики
Образовательная программа Интеллектуальные системы в гуманитарной сфере
Направление подготовки 45.03.04 Интеллектуальные системы в гуманитарной сфере

ОТЧЕТ

По курсовой работе дисциплины Мобильная разработка (Android и IOS) на тему
“Разработка Android-приложения Мессенджер”

Обучающийся:
Лазебный Всеволод Владимирович

Преподаватель:
Шатинский Григорий Сергеевич

Санкт-Петербург, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Обзор используемых технологий и архитектуры	4
1.1 Архитектура MVVM	4
1.2 Компоненты Android Jetpack	4
1.3 Неоморфный дизайн	5
2 Проектирование и реализация приложения	6
2.1 Навигация между экранами	6
2.2 Реализация экрана профиля	6
2.3 Реализация экрана настроек	7
2.4 Реализация экрана ленты сообщений	8
2.5 Реализация системы обновлений и уведомлений	10
2.6 Работа с разрешениями	11
2.7 Адаптивный дизайн	12
4 Тестирование приложения	12
ЗАКЛЮЧЕНИЕ	14

ВВЕДЕНИЕ

Данная курсовая работа посвящена разработке мобильного приложения под Android.

Цель заключается в разработке и исследовании Android-приложения с использованием современных технологий мобильной разработки, таких как: Android Jetpack, MVVM и других. Разрабатываемое приложение должно имитировать клиент мессенджера, в него должны быть включены окно с сообщениями, профилем и настройками. Основными действиями будут: изменение темы оформления, просмотр ленты сообщений и получение уведомлений о новых сообщениях.

Приложение должно быть разработано в среде Android Studio с версией операционной системы не ниже 13 с использованием таких языков как Kotlin, Java. В процессе реализации используются компоненты: LiveData, ViewModel, Navigation, Room, WorkManager и др. Для оформления была использована open-source библиотека из git - Android_neumorphic:1.2.0

Задачи:

- 1) Реализовать архитектуру приложения на основе MVVM,
- 2) Настроить навигацию между экранами,
- 3) Реализовать экран настроек со сменой темы,
- 4) Реализовать экран профиля с возможностью редактирования имени и почты,
- 5) Реализовать экран имитации ленты сообщений с отображением списка сообщений от пользователей и возможностью ставить лайк и обновлять страницу,
- 6) Реализовать имитацию периодической синхронизации с помощью WorkManager, отображать уведомления в системе устройства при получении новых сообщений,
- 7) Реализовать отправку запроса разрешения на отправку уведомлений при первом запуске приложения выше версии Android 6 и выше,
- 8) Реализовать обработку состояния подключения к сети.

1 Обзор используемых технологий и архитектуры

1.1 Архитектура MVVM

Архитектура MVVM (Model-View-ViewModel), рекомендованная Google для Android-разработки, была выбрана как оптимальное решение для создания поддерживаемого и тестируемого кода. В этой архитектуре Model отвечает за данные и бизнес-логику, View представляет пользовательский интерфейс, а ViewModel выступает посредником между ними, обеспечивая реактивное обновление данных. На Рисунке 1 представлена общая схема взаимодействия компонентов в приложении.

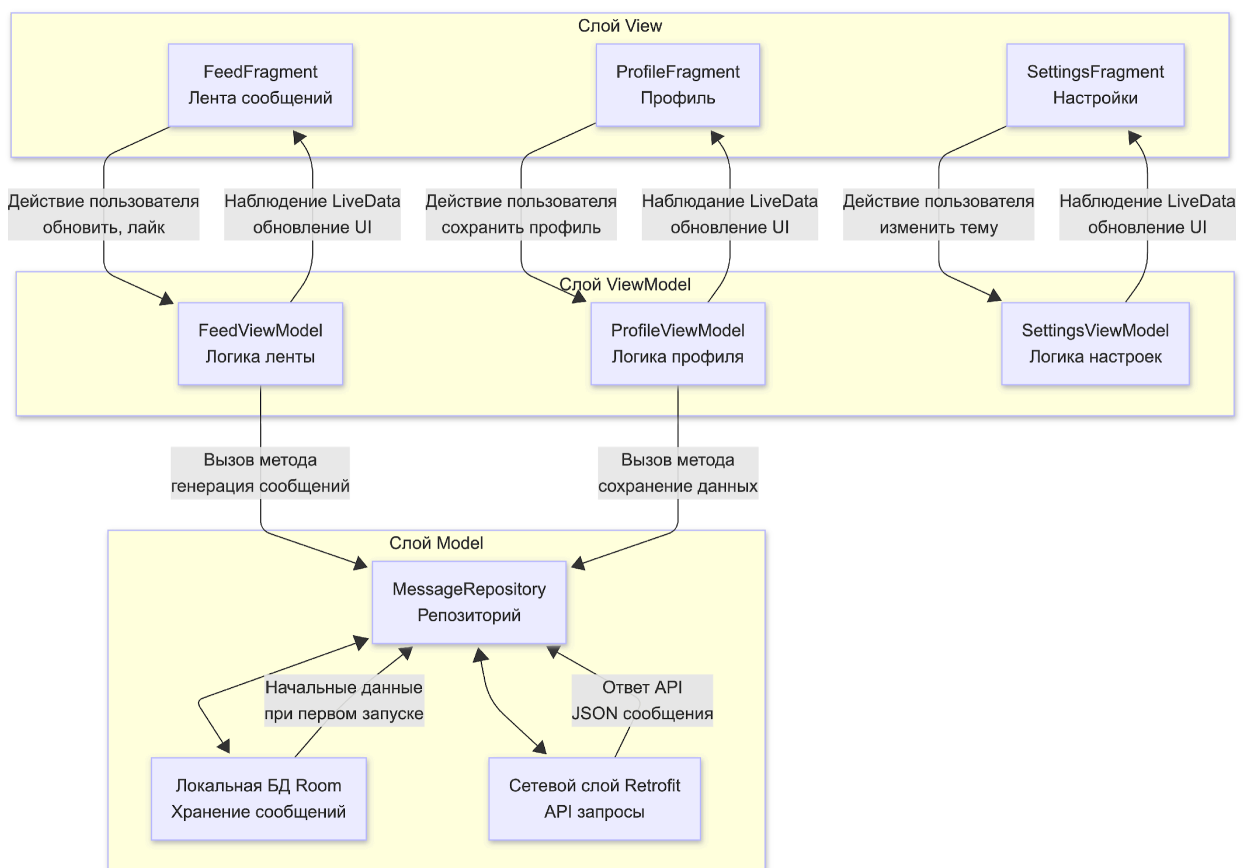


Рисунок 1 – общая схема компонентов

1.2 Компоненты Android Jetpack

В проекте активно используются компоненты Android Jetpack. LiveData обеспечивает реактивное обновление UI при изменении данных, ViewModel сохраняет состояние при повороте экрана, Room предоставляет абстракцию над SQLite для локального хранения данных, WorkManager управляет

фоновыми задачами, а Navigation Component упрощает навигацию между экранами. Эти компоненты интегрированы в единую систему, что соответствует современным стандартам.

1.3 Неоморфный дизайн

В качестве основы пользовательского интерфейса была выбрана неоморфная стилистика дизайна. Этот современный тренд в UI/UX характеризуется мягкими тенями и объемами, создающими иллюзию физических объектов. Для реализации неоморфизма была использована библиотека `Android_neumorphic`, взятая из `opensource` репозитория предоставляющая готовые компоненты с эффектом объемности. Например, в макете `fragment_profile.xml` карточка профиля реализована с помощью `RoundRectView`, который настраивается через атрибуты `light_color` и `dark_color` для создания эффекта освещения и теней (см. Рисунок 2).

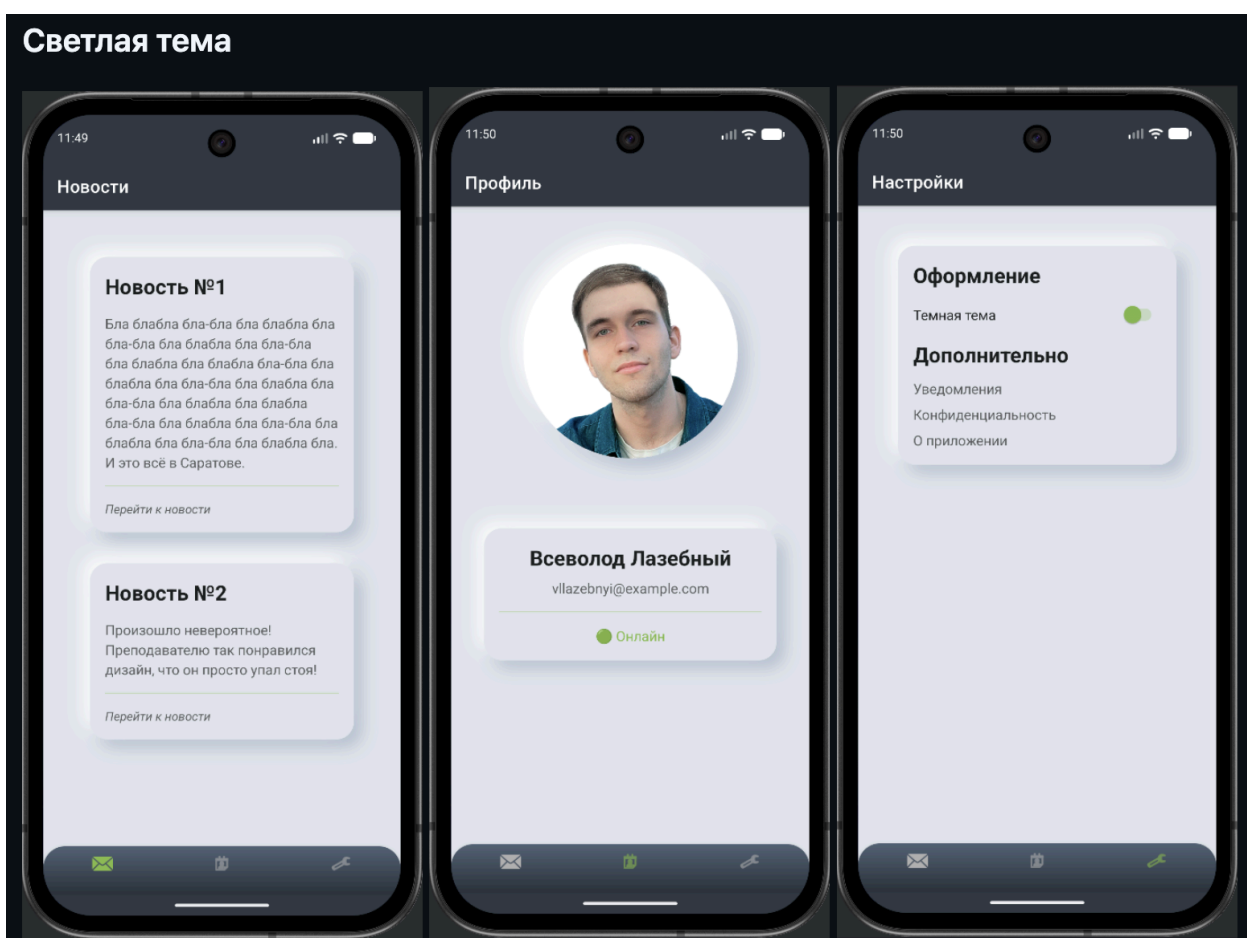


Рисунок 2 – Неоморфный дизайн из лабораторной №1

2 Проектирование и реализация приложения

2.1 Навигация между экранами

Навигация в приложении организована с помощью Navigation Component. Создан файл графа навигации `nav_graph.xml`, в котором определены три основных фрагмента: `FeedFragment` для ленты сообщений, `ProfileFragment` для профиля пользователя и `SettingsFragment` для настроек. Эти фрагменты связаны с `BottomNavigationView`, что позволяет пользователю переключаться между ними одним нажатием (см. Рисунок 3).

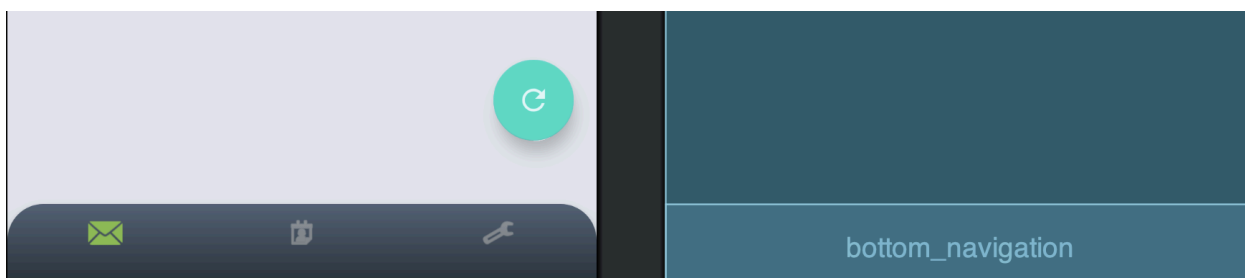


Рисунок 3 – Элемент навигации между экранами

В `MainActivity` происходит инициализация `NavController` и его связывание с нижней панелью навигации.

2.2 Реализация экрана профиля

Экран профиля реализован в `ProfileFragment` с соответствующим `ViewModel`. Пользователь может редактировать свое имя и email, которые сохраняются в `MutableLiveData` внутри `ProfileViewModel`. При нажатии кнопки "Сохранить изменения" новые данные передаются во `ViewModel`, которая обновляет `LiveData`. `Fragment` наблюдает за этими `LiveData` через `Observer` и автоматически обновляет интерфейс. Для отображения аватарки используется `ImageView` с загрузкой изображения через `Glide`, при этом реализована заглушка `default_avatar.png` на случай ошибки загрузки. На Рисунке 4 представлен финальный вид экрана профиля.

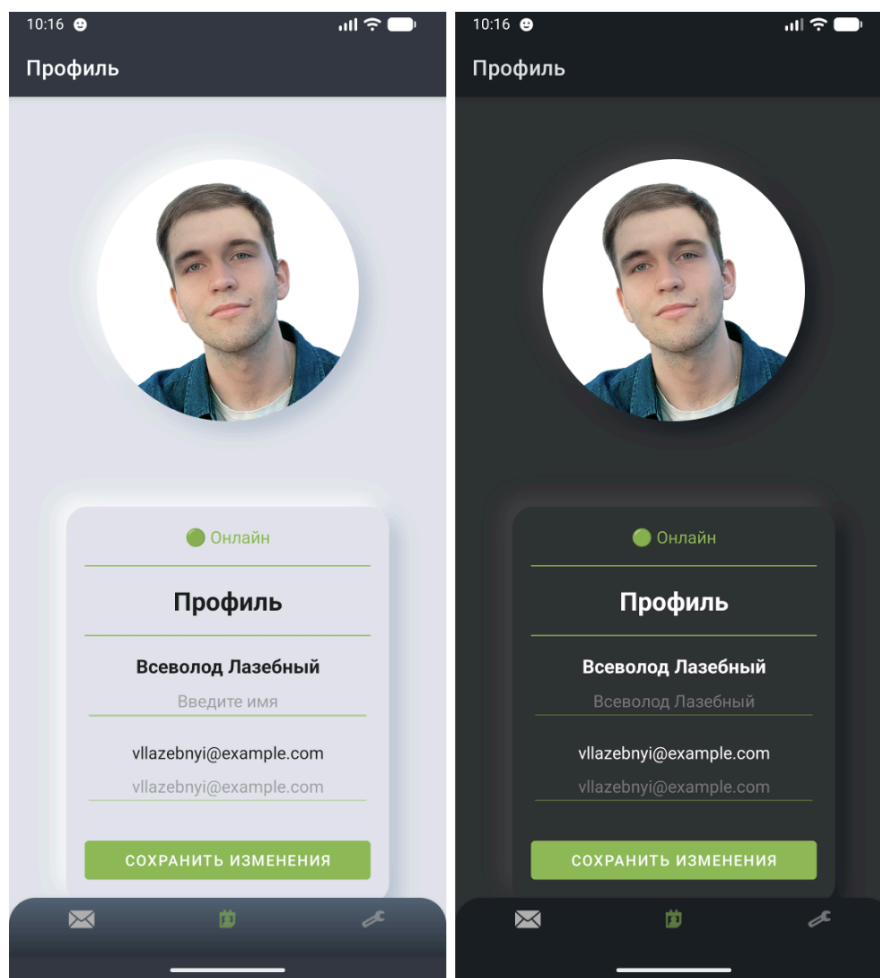


Рисунок 4 – Экран профиля

2.3 Реализация экрана настроек

SettingsFragment предоставляет возможность переключения между светлой и темной темой. Состояние переключателя сохраняется в SharedPreferences через SettingsViewModel, что позволяет сохранять выбор пользователя между запусками приложения. При изменении положения переключателя вызывается AppCompatActivity.setDefaultNightMode(), который мгновенно применяет новую тему. Также на экране настроек реализован переключатель для включения/выключения уведомлений. На Рисунке 5 представлен финальный вид экрана настроек.

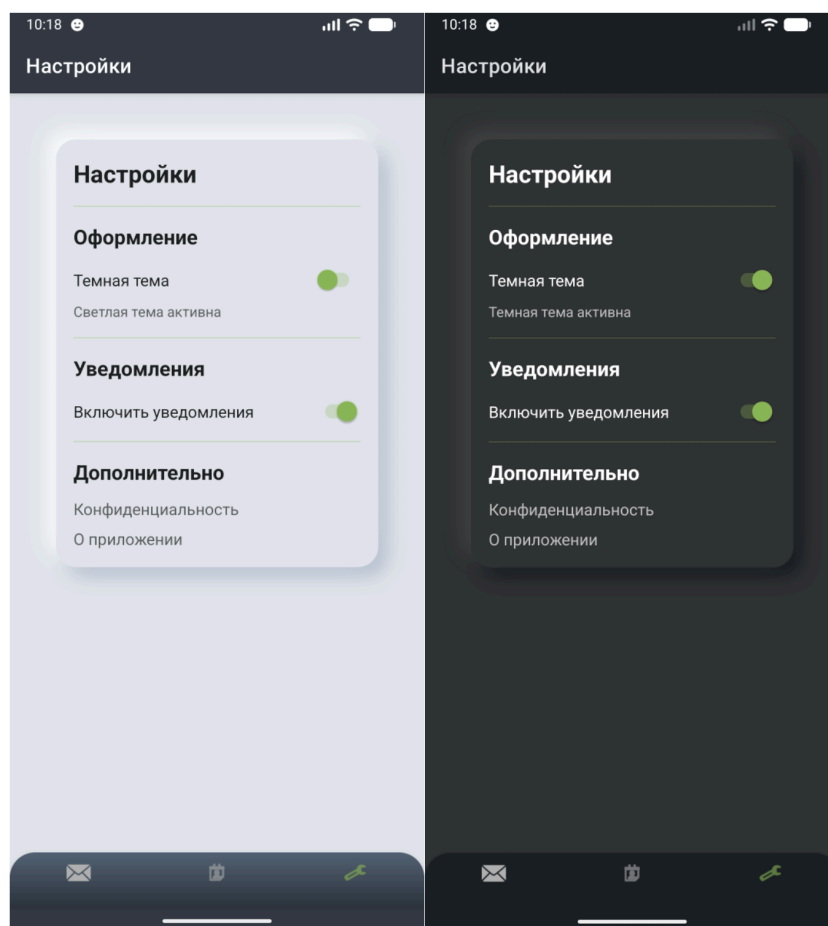


Рисунок 5 – Экран настроек

2.4 Реализация экрана ленты сообщений

Структура данных приложения организована с использованием Room. Сущность Message определена как Entity с полями id, title, body, userId, а также дополнительными полями для расширенной функциональности: userName, avatarUrl, isLiked и likeCount. Для управления данными создан интерфейс MessageDao с методами для получения, вставки и обновления сообщений.

Важной особенностью проекта стала реализация локальной генерации сообщений вместо использования полноценного API. Это решение было принято из-за отсутствия вменяемого доступного сервиса для тестирования и времени для написания своего API. В классе MessageRepository реализован метод generateNewMessages(), который создает сообщения с использованием

случайных данных: имен пользователей из предопределенного массива, аватарок с сервиса pravatar.cc и текстовых шаблонов.

Экран ленты сообщений использует `RecyclerView` с кастомным адаптером `EnhancedMessageAdapter`. Для оптимизации работы со списком применен механизм `DiffUtil`, который вычисляет разницу между старым и новым списком сообщений и анимирует только измененные элементы. Адаптер поддерживает загрузку аватарок через библиотеку `Glide` с показом заглушки `default_avatar.png` в случае ошибки загрузки.

На Рисунке 6 представлен финальный вид экрана главной страницы с лентой сообщений.

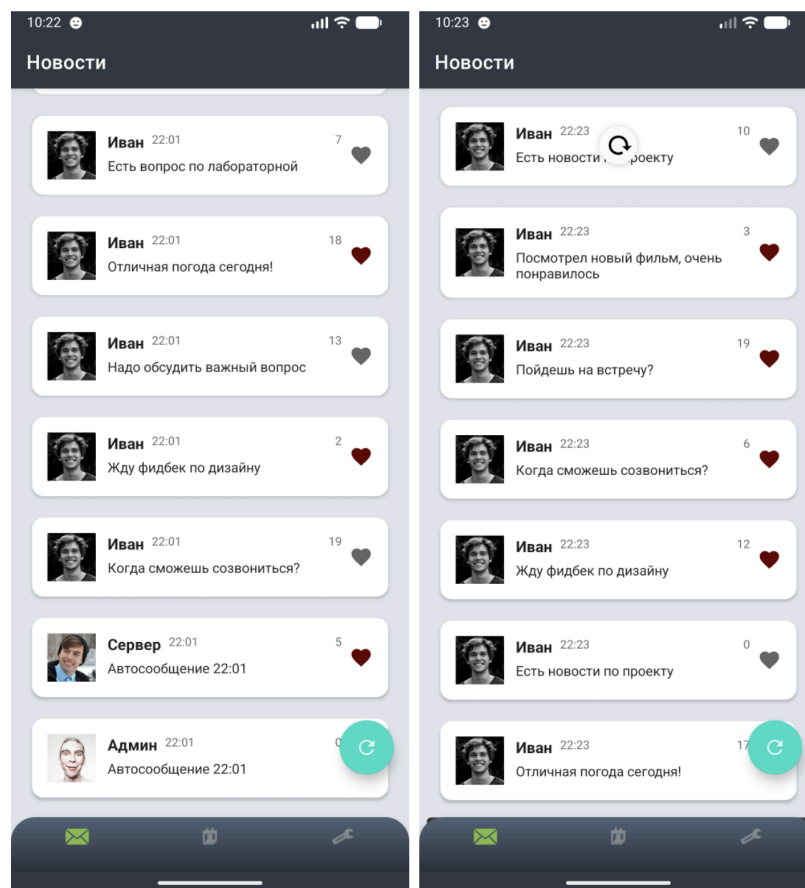


Рисунок 6 – Главный экран и лента сообщений

Система лайков реализована с учетом уникальности каждого сообщения. Каждый `Message` получает уникальный ID через `UUID.randomUUID().toString()`, а метод `toggleLike()` в репозитории корректно обновляет состояние только конкретного сообщения в базе данных. При

нажатии на иконку лайка происходит инкремент или декремент счетчика и изменение состояния isLiked.

2.5 Реализация системы обновлений и уведомлений

Реализована гибридная система обновлений. Пользователь может вручную обновить ленту двумя способами: свайпом вниз (через SwipeRefreshLayout) или нажатием на плавающую кнопку обновления. В обоих случаях генерируется 2 новых сообщения (см. Рисунок 7).

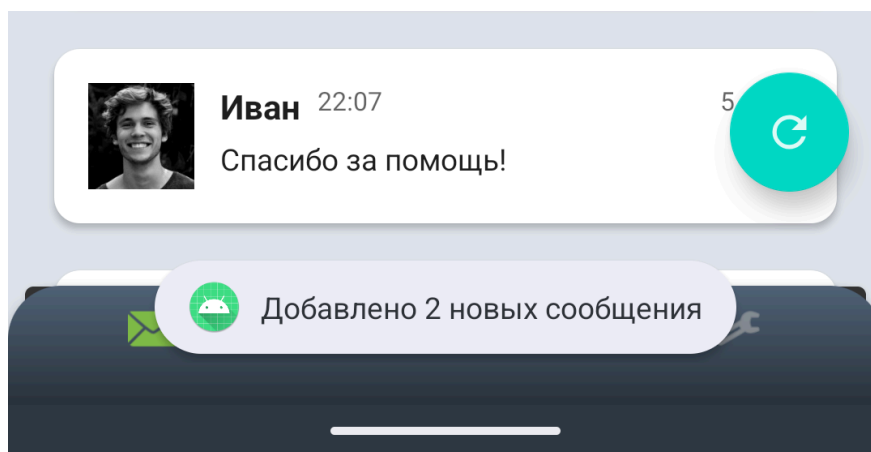


Рисунок 7 – Уведомление о 2 новых сообщениях после ручного запроса

Параллельно работает автоматическая система: каждые 20 секунд в фоне создается 5 новых сообщений, о чем пользователь уведомляется через системное уведомление.

Для фоновой синхронизации данных применен WorkManager. Класс SyncWorker наследует от Worker и переопределяет метод doWork(), в котором происходит генерация случайного сообщения и показ уведомления. WorkManager настраивается на периодический запуск каждые 15 минут при наличии интернет-соединения.

Система уведомлений о новых сообщениях реализована через NotificationCompat. При автоматической генерации 5 сообщений создается канал уведомлений (для Android 8.0 и выше) и показывается уведомление с

заголовком "Новые сообщения!" и текстом о количестве добавленных сообщений. На Рисунке 8 показан пример такого уведомления.

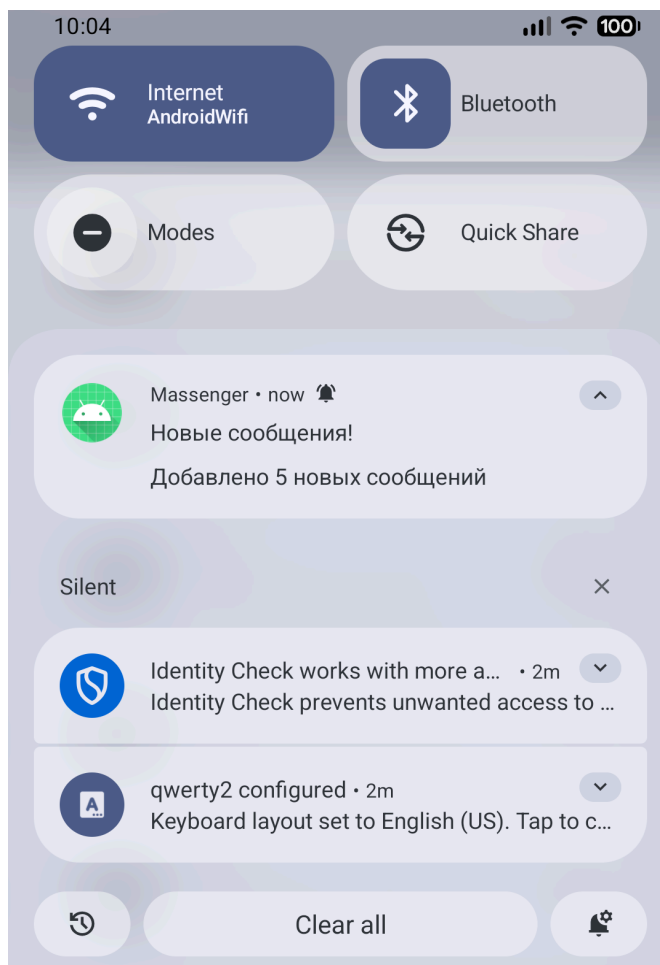


Рисунок 8 – Уведомление о 5 новых сообщениях при авто обновлении

2.6 Работа с разрешениями

Особое внимание было уделено обработке разрешений для уведомлений. Начиная с Android 13, для показа уведомлений требуется явное разрешение пользователя. В методе `requestPermissions()` `FeedFragment` проверяет наличие разрешения `POST_NOTIFICATIONS` и запрашивает его при необходимости через системный диалог. Также реализована проверка состояния сети через `ConnectivityManager` для обеспечения корректной работы в условиях отсутствия интернет-соединения. На Рисунке 9 представлен финальный вид запроса на разрешение уведомлений приложением.

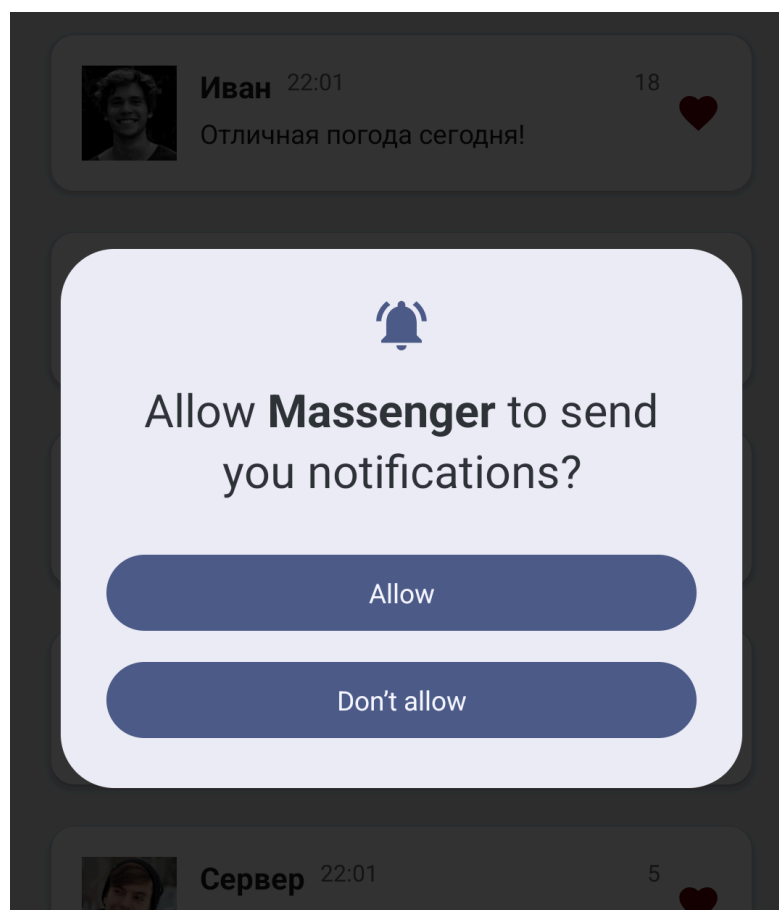


Рисунок 9 – Запрос разрешения уведомлений в системе

2.7 Адаптивный дизайн

Для поддержки разных ориентаций экрана созданы отдельные макеты в папках `layout` и `layout-land`. Например, в альбомной ориентации экран профиля разделен на две колонки: слева размещается аватар, справа - редактируемые поля (см. Рисунок 10). Это улучшает пользовательский опыт.

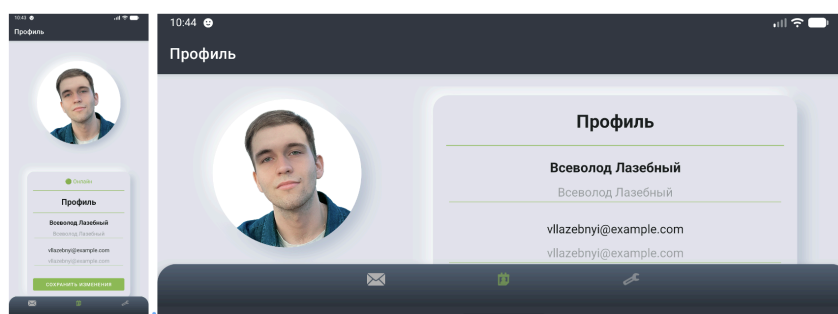


Рисунок 10 – Пример адаптивности дизайна

4 Тестирование приложения

В процессе разработки были решены несколько технических проблем. Одна из них касалась корректного отображения времени сообщений. В классе `EnhancedMessageAdapter` реализован метод `formatTime()`, который обрабатывает различные форматы времени. Другая проблема была связана с прокруткой к новым сообщениям - реализована логика, которая прокручивает список к началу только если пользователь уже находится в верхней части списка. Приложение тестировалось на эмуляторах устройств с разными версиями Android.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано полнофункциональное Android-приложение, имитирующее клиент мессенджера. Успешно реализованы все поставленные задачи: внедрена архитектура MVVM, настроена навигация между экранами, созданы экраны профиля, настроек и ленты сообщений с системой лайков, реализована фоновая синхронизация и система уведомлений. Проект демонстрирует решение, которое может служить основой для дальнейшего развития в полноценный мессенджер с реальным бэкендом.

Значительный опыт был получен в работе с пользовательским интерфейсом. Были освоены Material Design компоненты, включая RecyclerView для эффективного отображения списков сообщений, SwipeRefreshLayout для реализации жеста обновления свайпом, FloatingActionButton для кнопки обновления. Особый интерес представляла реализация поддержки различных конфигураций: создание отдельных макетов для портретной и альбомной ориентации, реализация переключения между светлой и темной темами с сохранением выбора пользователя. Отдельного упоминания заслуживает опыт работы с неоморфным дизайном через библиотеку Android_neumorphic, что позволило создать уникальный визуальный стиль приложения с объемными элементами и мягкими тенями.

Были изучены механизмы работы с системными функциями Android: реализация уведомлений с созданием каналов уведомлений для Android 8.0 и выше, запрос runtime-разрешений для показа уведомлений начиная с Android 13, работа с SharedPreferences для сохранения пользовательских настроек. Важным аспектом стала организация работы с данными: освоены принципы создания репозитория как единого источника данных, работа с базой данных, реализация механизма DiffUtil для оптимизации обновления RecyclerView.